Anno Accademico 2026/2027

Leonardo Donati

Appunti

Sistemi Operativi Avanzati

Teoria e esercizi.



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

Indice

1 Introduzione **3**
- 1.1 Sistema Operativo . . . 3
- 1.2 Organizzazione del Sistema di un Computer . . . 4
  - 1.2.1 Interrupt . . . 4
  - 1.2.2 Struttura della Memoria . . . 6
- 1.3 Architetture dei Computer . . . 6
- 1.4 Operazioni di un Sistema Operativo . . . 6
  - 1.4.1 Multitasking e Multiprogramming . . . 7
  - 1.4.2 Operazioni Dual-Mode . . . 7
  - 1.4.3 Timer . . . 7
- 1.5 Sicurezza e Protezione . . . 7
- 1.6 Struttura di un Sistema Operativo . . . 8

2 Chiamate di Sistema **9**
- 2.1 Application Programming Interface (API) . . . 9
  - 2.1.1 Tipi di Chiamate di Sistema . . . 10
- 2.2 Servizi di Sistema . . . 11
- 2.3 Linker e Loaders . . . 11

3 Processi **13**
- 3.1 Stato dei Processi . . . 13
- 3.2 Process Control Block (PCB) . . . 14
- 3.3 Scheduling dei Processi . . . 15
  - 3.3.1 Code di Scheduling . . . 15
  - 3.3.2 Cambio di Contesto . . . 16
- 3.4 Creazioni di Processi . . . 16
  - 3.4.1 System Calls `fork()` e `exec()` . . . 17
  - 3.4.2 Terminazione dei Processi . . . 17
- 3.5 Comunicazione fra Processi . . . 18
  - 3.5.1 Shared Memory . . . 18
  - 3.5.2 Messagge Passing . . . 19
  - 3.5.3 Sincronizzazione . . . 20
  - 3.5.4 Buffering . . . 20
  - 3.5.5 Esempi POSIX . . . 20

Capitolo 1

Introduzione

1.1 Sistema Operativo

> **DEFINITION** Sistema Operatvio
>
> Un sistema operatvio è un **software** che agisce come intermediario fra l'hardware e l'utente.

Ha diversi obiettivi:

- Eseguire programmi dell'utente.
- Far diventare l'uso del dispositivo conveniente.
- Usare l'hardware del computer in modo efficiente.

Per capire bene cosa fa un sistema opertivo è utili dividere la struttura di un computer in quattro parti principali:

1. **Hardware:** provvede le risorse basi, come la cpu, la memoria centrale e i dispositivi di I/O.
2. **Sistema operativo:** controlla e gestisce l'hardware per applicazioni e utenti.
3. **Programmi applicativi:** decide come le risorse hardware devono essere divise fra i vari programmi.
4. **Utenti:** agenti esterni che interagiscono con il computer.

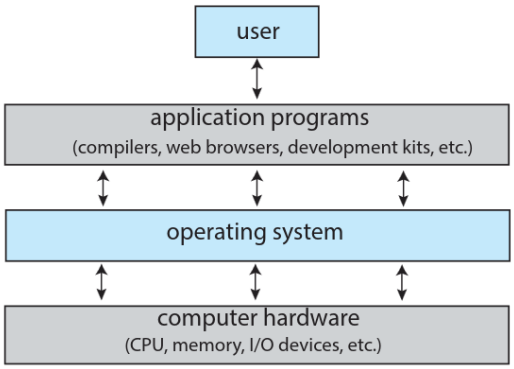


Figura 1.1: Architettura del sistema di un computer.

Dal punto di vista di un utente, deve essere facile usare applicazioni e avere buone prestazioni.
Dal punto di vista del computer, però, tutti gli utente devono essere soddisfatti, quindi il sistema operativo deve essere un **allocatore di risorse** e un **gestore di programmi**.

1.2 Organizzazione del Sistema di un Computer

Un computer moderno **general-purpose** consiste in una o più CPU e un numero di dispositivi che sono collegati tramite un **bus** comune che da accesso alla memoria condivisa e agli altri componenti.

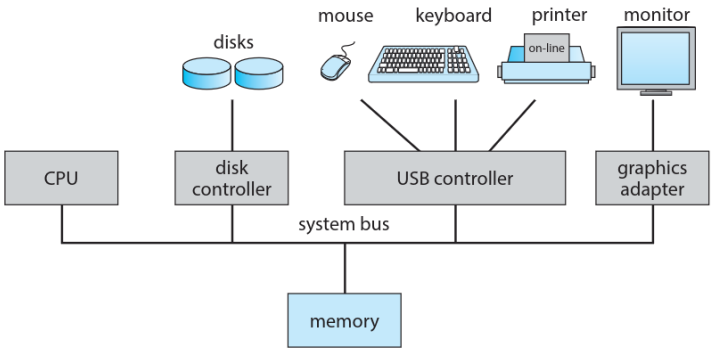


Figura 1.2: Organizzazione del sistema di un computer.

Ogni dispositivo ha un controller, che si occupa di gestire i dati fra il suo buffer interno e altri componenti. I sistemi operatvi per interagire in modo uniforme con i controller, usano dei **driver**. Dato che la CPU e i controller possono agire in parallelo, competendo per cicli di memeoria, bisogna trovare un modo ordinato per l'accesso alla memoria condivisa.

1.2.1 Interrupt

EXAMPLE Operazione di I/O

Per un'operazione di I/O si succedono diverse azioni:

- Il driver carica correttamente i registri del controller.
- Il controller guarda i regsitri per capire che operazione fare.
- Il controller carica i dati da un dispositivo al suo buffer locale.
- Il controller informa il driver di aver finito l'operazione.

Per ultimo, il driver da il controllo al sistema operativo, passando i dati letti o un puntatore a questi.

Il controller per avvisare il driver di aver finito la sua operazione, usa una **interrupt**.

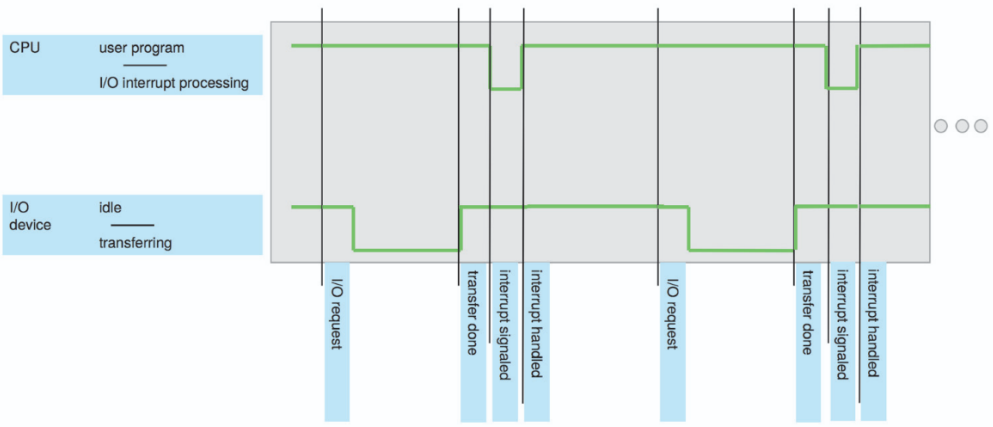


Figura 1.3: Timeline di una interrupt.

Quando la CPU riceve una interrupt, ferma immediatamente quello che sta facendo e trasferisce l'esecuzione alla service routine corretta, fino a che non è stata completata. Una volta finita, la CPU ricomincia da dove aveva interrotto, inserendo nel **program counter** l'indirizzo corretto.

> **NOTA BENE**
>
> Questa metodologia richiede, però, di dover salvare lo stato delle informazioni della CPU, dato che le service routine degli interrupt potrebbero modificare i registri.

Le interrupt, dato che sono molto frequenti, devono essere gestite velocemente. Per questo, si può usare una tabella, salvata nella parte basse della memoria, che contiene tutti gli indirizzi delle service routine degli interrupt, garantendo la velocità necessaria. Questo array (o **vettore degli interrupt**) di indirizzi, ha come indici dei numeri univoci che specificano la routine corretta per il tipo di interrupt.

vector number	description
0	divide error
1	debug exception
2	null interrupt
3	breakpoint
4	INTO-detected overflow
5	bound range exception
6	invalid opcode
7	device not available
8	double fault
9	coprocessor segment overrun (reserved)
10	invalid task state segment
11	segment not present
12	stack fault
13	general protection
14	page fault
15	(Intel reserved, do not use)
16	floating-point error
17	alignment check
18	machine check
19–31	(Intel reserved, do not use)
32–255	maskable interrupts

Figura 1.4: Tabella degli indici di un vettore di interrupt.

Implementazione

L'hardware della CPU ha un cavo chiamato **interrupt-request line**, dove la CPU riceve i segnali dai controller. Una volta intercettato il numero dell'interruzione, tramite il vettore degli interrupt, viene chiamata l'**interrupt-handler routine** corretta.
In sistemi operativi moderni, il metodo deve essere più sofisticato:

- È necessaria la possibilità di differire la gestione delle interruzioni durante l'esecuzione di elaborazioni critiche.
- È necessario un metodo efficiente per indirizzare l'esecuzione all'interrupt-handler corretto per un determinato dispositivo.
- Sono necessarie interruzioni multilivello, in modo che il sistema operativo possa distinguere tra interruzioni ad alta e bassa priorità e rispondere con il livello di urgenza adeguato.

Queste tre caratteristiche sono fornite dalla CPU e dall'**interrupt-controller hardware**.

Per gestire la priorità tra interrupt, nelle CPU esistono due linee di richieste per le interruzioni:

1. **Maskable:** può essere momentaneamente spenta dalla CPU, per dare priorità a istruzioni critiche. Questa linea è usata dai controller dei dispotivi.
2. **Non-Maskable:** usata per interrupt che non possono essere ignorate, come errori di memoria non recuperabili.

1.2.2 Struttura della Memoria

La memoria di un computer è organizzata in un gerarchia che tiene conto di tre valori:

1. **Velocità**.
2. **Capacità**.
3. **Volatilità**.

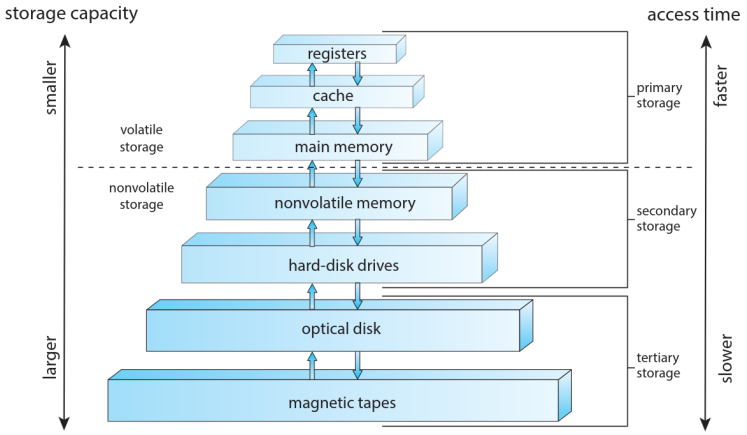


Figura 1.5: Gerarchia delle memorie.

Per spostare grandi quantità di dati, la metodologia degli interrupt descritta prima potrebbe creare molto overhead, per questo viene usata la **DMA** (**Direct Memory Access**).

> **DEFINITION** Direct Memory Access
>
> Dove aver sistemato buffer e puntatori, il controller del dispositivo sposta un grande blocco di dati dalla memoria al componente, o viceversa, senza l'intervento della CPU. Questo genera una sola interrupt per blocco, a discapito delle numerose interrupt generate senza DMA.

1.3 Architetture dei Computer

Possono esistere diverse architetture, basate sul numero di processori presenti all'interno del computer:

- **Single-Processor:** se è presente un solo processore.
- **Multi-Processor:** attualmente comune come **Multi-Core**, se sono presenti più processori. Condividono la memoria centrale e i dispositivi di I/O.
- **Cluster:** si distinguono dai computer multi-processor, perchè sono sistemi autonomi che lavorano insieme. Ogni nodo è spesso un computer multi-core.

> **DEFINITION** Alcune concetti di base
>
> **CPU:** hardware che esegue le istruzioni.
> **Processor:** chip che contiene una o più CPU.
> **Core:** unità basica della CPU.

1.4 Operazioni di un Sistema Operativo

Per avviare il computer, è necessario un programma di iniziale: il **bootstrap**. Serve a inizializzare tutte le risorse utili, come registri e i vari controller. In ultimo, deve localizzare il kernel del sistema operativo e caricarlo in memoria.

Il sistema operativo, in assenza di azione da eseguire, aspetta degli eventi, come le interrupt a livello hardware o le **trap** (**exceptions**) a livello software.

1.4.1 Multitasking e Multiprogramming

L'obiettivio del sistema operativo è tenere la CPU occupata tutto il tempo e nel mentre garantire all'utente di utilizzare più programmi contemporaneamente. Tutto questo è fattibile grazie alla capacità di essere multiprogramming, potendo gestire più programmi in esecuzione o **processi**.
Invece, il multitasking, è la capacità di poter saltare da un processo all'altro velocemente, garantendo un tempo di risposta molto basso all'utente.

1.4.2 Operazioni Dual-Mode

PROBLEMA

Il sistema operativo e gli utenti risiedono sullo stesso hardware. Si deve trovare una soluzione per evitare di eseguire codice malevolo che potrebbe alterare il funzionamento di altri programmi o del sistema stesso.

Come minimo, sono necessarie due modalità: **Kernel Mode** e **User Mode**. All'hardware è stato aggiunto un bit, chiamato **bit mode**, che permette di distinguere la modalità corrente: kernel (0) e user (1).

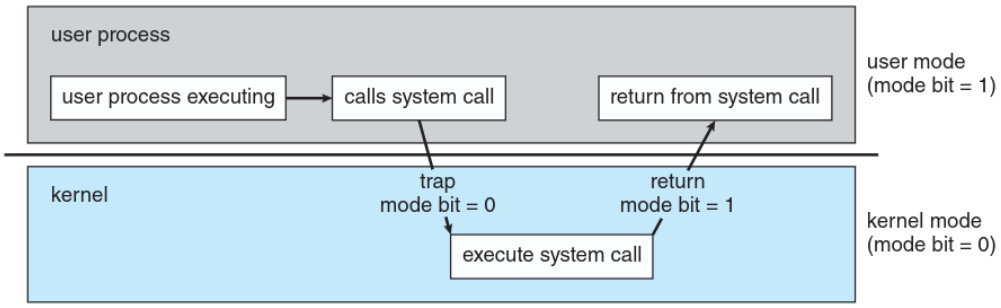


Figura 1.6: Operazione Dual-Mode.

Se si prova a eseguire una istruzione vietata in user mode, allora l'hardware lancerà una trap al sistema operativo, il quale tramite il vettore delle interrupt, prende il controllo e termina l'esecuzione del processo.

1.4.3 Timer

Bisogna assicurarsi che il sistema operativo mantenga sempre il controllo sulla CPU, per evitare che programmi in esecuzione dell'utente si blocchino in un loop infinito, oppure che una system call fallita non ritorni il controllo al sistema operativo.
Una soluzione è usare un **timer**, che interrompe il computer dopo un certo periodo. Solitamente, viene

implementato tramite un clock a frequenza fissa e un contatore. Ogni ciclo di clock, il contatore viene diminuito di una unità.

TIMER SU SISTEMI LINUX

Nei sistemi Linux, il parametro di configurazione del kernel `HZ` specifica la frequenza delle interrupt del timer. Per esempio, `HZ` = 250, significa 250 interruzioni al secondo o una ogni 4 millisecondi.

1.5 Sicurezza e Protezione

Nei sistemi operativi moderni, che supportano più utenti, i dati che vengono usati devono essere garantita solo al processo che ha ricevuto tutte le autorizzazioni.

DEFINITION Protezione

Qualsiasi meccanismo per controllare l'accesso alle risorse di un computer da parte di utenti o processi. Deve essere possibile avere un modo per specificare i controlli e controllare che vengano rispettati.

Anche se è presente un alto grado di protezione, può rimanere comunque vulnerabile a fallimenti o accessi non autorizzati.

> **DEFINITION** Sicurezza
>
> Difendere il sistema da attacchi interni ed esterni, come dos, worms, furti di dati e identità, ecc...

I sistemi operativi per distinguere fra gli utenti, usa diversi identificativi:

- **User ID:** nome e numero associato a ogni utente.
- **Group Identifier:** per definire un gruppo di utenti e associargli risorse e processi.

Per ottenere più permessi, è possibile fare **privilage escalation** per cambiare il proprio ID e avere più permessi.

1.6 Struttura di un Sistema Operativo

Capitolo 2

Chiamate di Sistema

DEFINITION Chiamata di Sistema

Interfaccia ai servizi principali, messa a disposizione da il sistema operativo. Solitamente, queste chiamate sono funzioni scritte in C/C++ o per alcuni casi specifici in linguaggio Assembly.

2.1 Application Programming Interface (API)

Nei sistemi moderni, vengono eseguite migliaia di system calls ogni secondo, quindi, gli sviluppatori, per realizzare le applicazioni si affidano a una **Application Programming Interface** (**API**).

DEFINITION API

Insieme di funzioni disponibili per un programmatore di applicazioni, inclusi i parametri passati a ciascuna funzione e i valori di ritorno che il programmatore può aspettarsi.

Le API più comuni sono quella di Windows, POSIX (Unix, Linux e MacOS) e quella per la Java Virtual Machine.

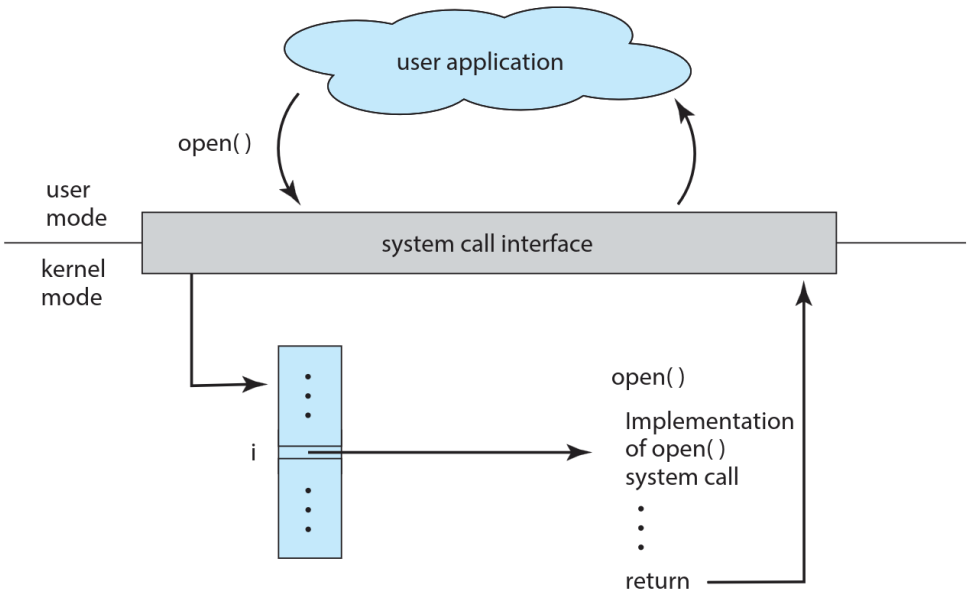


Figura 2.1: Esempio di System Call.

Un fattore nella gestione delle chiamate di sistema è l'**ambiente di esecuzione** (**RTE**), ossia l'insieme di software necessari per eseguire applicazioni scritte in un determinato linguaggio di programmazione, inclusi i relativi compilatori o interpreti, oltre ad altri componenti come librerie e loader.

L'RTE, tramite una **system call interface**, fa da link fra le system call al sistema operativo e le istruzione dell'API.
Tipicamente, a ogni chiamata di sistema viene associato un numero e viene mantenuta una tabella indicizzati secondi questi valori per gestire le call.

I parametri delle system call, vengono passati tramite registri, nel caso non sia possibile, tramite blocchi memoria oppure con una push sullo stack, dove poi il sistema oprativo eseguirà una pop.

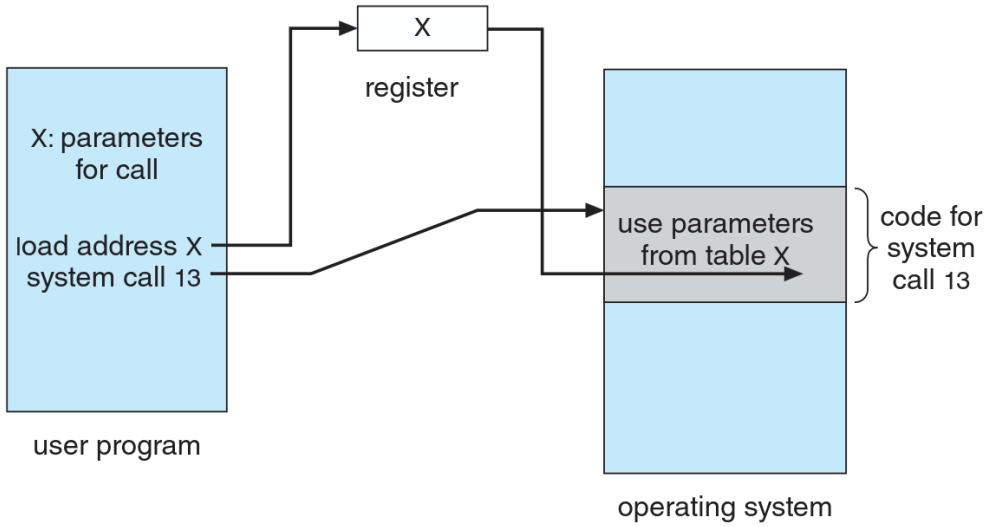


Figura 2.2: Esempio di Passaggio di Parametri.

2.1.1 Tipi di Chiamate di Sistema

Esistono sei tipi di system calls:

1. **Process Control**:
   (a) Creazione e terminazione di processi;
   (b) Caricamento ed esecuzione;
   (c) Eventi e segnali di wait;
   (d) Allocazione e liberazione della memoria;
   (e) Get e set di attributi dei processi.
2. **File Management**
   (a) Creazione ed eliminazione di file;
   (b) Apertura e chiusura di file;
   (c) Lettura e scrittura di file;
   (d) Get e set di attributi dei file.
3. **Device Management**:
   (a) Richiesta e rilascio un dispositivo;
   (b) Lettura e scrittura;
   (c) Get e set degli attributi di un dispositivi;
   (d) Attach e detach logico di dispositivi.
4. **Information Maintenance**:
   (a) Get e set di tempo e data;
   (b) Get e set di dati di sistema;
   (c) Get e set di attributi di file, processi e dispositivi.

5. **Communications**:
   (a) Creazione ed eliminazione canali di comunicazione;
   (b) Invio e ricezione di messaggi;
   (c) Trasferimento di informazioni di stato;
   (d) Attach e detach di dispositivi remoti.
6. **Protection**:
   (a) Get file permission;
   (b) Set file permission.

2.2 Servizi di Sistema

> **DEFINITION** Servizi di Sistema
>
> Offrono un ambiente conveniente per programmare ed eseguire applicazioni.
> Alcuni sono solamente warp di chiamate di sistema, altre invece sono più complessi.

Ci sono diverse categorie:

- **Gestione dei file:** permettono di eseguire diverse operazioni su file e directory.
- **Informazioni di Status:** permettono di interrogare l'SO per avere informazioni come data, tempo, spazio su memoria o dischi, ecc...
  Inoltre, permettono azioni più complesse come il logging, il debugging e l'analisi di prestazioni.
- **Modifica dei File:** modificare o creare file su dischi, tramite, per esempio, svariati text editor.
- **Supporto a linguaggi di Programmazione:** Compilatori, debugger e interpreti per linguaggi di programmazione tipo C/C++, Java, Python, ecc...
- **Esecuzione e Caricamento dei Programmi:** i programmi devono essere caricati in memoria ed eseguiti. Il sistema deve fornire linker e loader, oltre a i vari debugger.
- **Comunicazioni:** permettono di creare canali di comunicazioni virtuali fra processi, utenti o computer.
- **Servizi di Bakcground:** alcuni servizi vengono lanciati nel tempo di boot, alcuni terminano, altri invece continuano a lavorare finchè il sistema non viene spento.
  Normalmente, quest'utlimi sono chiamati **demoni**.

2.3 Linker e Loaders

Solitamente, un programma per essere eseguito da una CPU deve essere caricato nella memoria centrale e metto in lista come processo. Per fare questa cosa sono necessari diverse fasi.

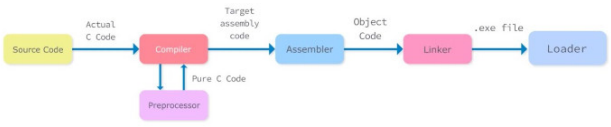


Figura 2.3: Ciclo completo di un programma C.

Per prima cosa, il programma viene compilato in un file oggetto per essere caricato in una memoria fisica. Questo formato è chiamato **file oggetto reallocabile**.

Successivamente, il **linker** combina i vari file oggetto e le librerie necessarie in un unico file eseguibile. Infine, il **loader** si occupa di caricare il programma in memoria centrale per essere eseguito dalla CPU. Un'attività importante di questa fase è la **riallocazione**, dato che è necessario assegnare un indirizzo finale alle varie parti del programma, ai dati e pure al codice.

I file oggetto e i file eseguibili presentano solitamente formati standard che includono il codice macchina compilato e una tabella dei simboli contenente metadati relativi a funzioni e variabili a cui il programma fa riferimento. Per i sistemi UNIX e Linux, questo formato standard è noto come **ELF** (**Executable and Linkable Format**).

EXAMPLE Formato ELF

Con il comando `file` è possibile identificare e valutare i formati ELF.
Per esempio con:

```
file main.o
```

verrà riportato che il `main.o` è un file ELF reallocabile.
Invece:

```
file main
```

riporterà che `main` è un file ELF eseguibile.

Capitolo 3

Processi

> **DEFINITION** Processo
>
> Un processo è un programma in esecuzione.

L'attività corrente del processo è rappresentata dal valore del **program counter** e dal contenuto dei registri. Il layout della memoria è diviso in varie parti:

- **Text Section:** codice eseguibile.
- **Data Section:** variabili globali.
- **Heap Section:** memoria allocata dinamicamente durante il tempo di esecuzione.
- **Stack Section:** spazio dei dati temporaneo quando vengono chiamate le funzioni.

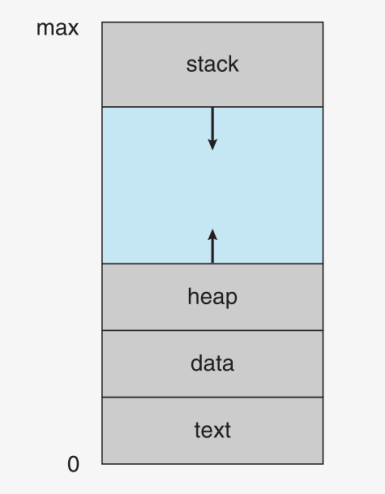


Figura 3.1: Layout Memoria.

La dimensione della sezione Data e della sezione Text sono fissate e non possono cambiare, invece, la dimensione dello Stack e dell'Heap può aumentare o diminuire dinamicamente.
Per esempio, per ogni funzione viene dichiarato un **Record di Attivazione**, con tutte i dati della funzione e viene pushato e tolto dallo stack. Invece, l'Heap può crescere in base alla memoria allocata.
Anche se queste due aree della memoria crescono una verso l'altra, il sistema operativo ha il compito di non farli mai accavallare.

3.1 Stato dei Processi

Durante la sua esecuzione, un processo può cambiare stato. Un processo può avere cinque stati differenti:

1. **New:** il processo è stato creato.

2. **Running:** le istruzione vengono eseguite.
3. **Waiting:** il processo sta aspettando l'accadimento di alcuni eventi.
4. **Ready:** il processo sta aspettando di essere assegnato a un processore.
5. **Terminated:** il processo ha finito l'esecuzione.

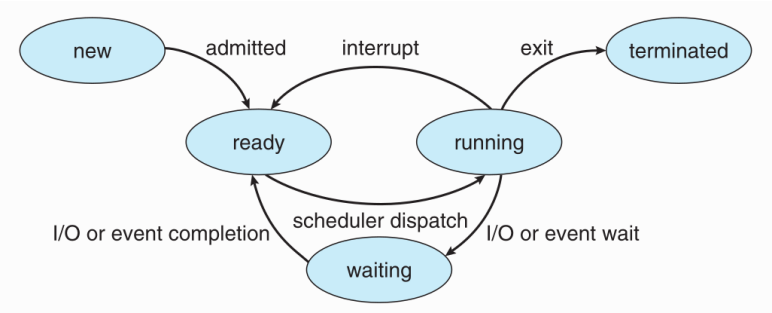


Figura 3.2: Stati Possibili di un Processo.

3.2 Process Control Block (PCB)

Ogni processo, all'interno del sistema operativo, è reppresentato da un **Process Control Block** o **Task Control Block**. Al suo interno sono contenuti tutte le informazioni associate a un processo.

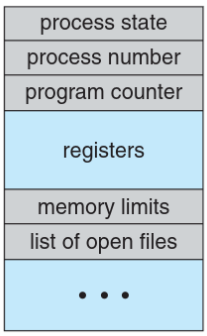


Figura 3.3: PCB.

Nello specifico, vengono specificate le seguenti informazioni:

- **Process State:** indica lo stato del processo.
- **Program Counter:** contiene l'indirizzo della prossima istruzione da eseguire.
- **CPU Registers:** contiene il tipo, il numero e lo stato dei registri usati.
- **CPU-Scheduling Information:** include i parametri per lo scheduling.
- **Memory-Managment Information:** pagine o tabelle di memoria e altre informazioni.
- **Accounting Informaztion:** quantità di tempo di CPU e tempo reale usato, job or process number, limiti di tempo e altre informazioni.
- **I/O Status Information:** lista dei dipositivi I/O parti, lista dei file aperti e altro.

All'interno dei sistemi Linux ogni PCB è reppresentato dalla structure `task_struct`, che si trova nel file `<include/linux/sched.h>` del kernel linux.

C struct task_struct

```c
struct task_struct {
  long state;                     /* state of the process */
  struct sched_entity se;         /* scheduling information */
  struct task_struct *parent;     /* this process's parent */
  struct list_head children;      /* this process's children */
  struct file_struct *files;      /* list of open files */
  struct mm_struct *mm;           /* address space */
}
```

Si può notare che all'interno della struct, oltre ai file aperti, lo stato del processo e altre informazioni, è presente pure il puntatore a una coda con i processi figli e uno che punta al processo padre.

Inoltre, sempre nei sistemi operativi Linux, sono presenti altri stati oltre a quelli classici:

- **TASK_INTERRUPTIBLE:** il task sta aspettando, può essere interrotto da un segnale.
- **TASK_UNINTERRUPTIBLE:** il task sta aspettando e non può essere interrotto da un segnale.
- **EXIT_ZOMBIE:** il processo è terminato ma le risorse devono ancora essere deallocate.
- **EXIT_DEAD:** il processo è terminato e le risorse sono state deallocate.
- **TASK_STOPPED:** il task è stato interrotto da una azione del terminale (`CTRL + Z`).
- **TASK_TRACED:** il task è seguito da un altro processo per un esecuzione passo passo.

C /include/linux/sched.h

```c
/* Used in tsk->__state: */
#define TASK_RUNNING 0x00000000
#define TASK_INTERRUPTIBLE 0x00000001
#define TASK_UNINTERRUPTIBLE 0x00000002
#define __TASK_STOPPED 0x00000004
#define __TASK_TRACED 0x00000008
#define EXIT_DEAD 0x00000010
#define EXIT_ZOMBIE 0x00000020
#define EXIT_TRACE (EXIT_ZOMBIE | EXIT_DEAD)

/* Used in tsk->__state again: */
#define TASK_PARKED 0x00000040
#define TASK_DEAD 0x00000080
#define TASK_WAKEKILL 0x00000100
#define TASK_WAKING 0x00000200
#define TASK_NOLOAD 0x00000400
#define TASK_NEW 0x00000800
```

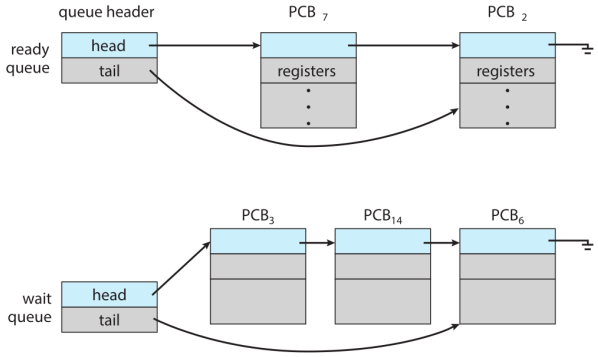
3.3 Scheduling dei Processi

Per una CPU con un solo core, non potrà mai esserci più di un processo in esecuzione. Invece, in caso di una archiettura multicore, i processi dovranno essere schedulati e aspettare un core libero. Il numero di processi in memoria è detto **grado di multiprogrammazione**.

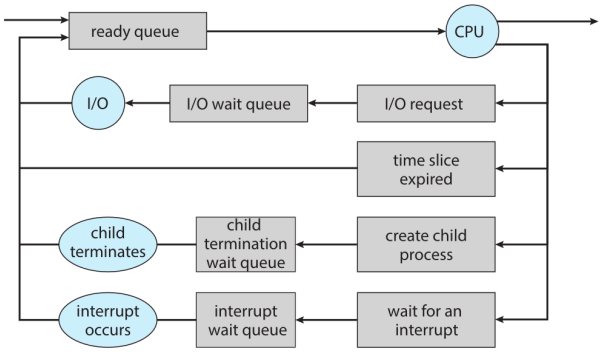
3.3.1 Code di Scheduling

Quando i processi entro nel sistema, vengono inseriti in una coda, la **Ready Queue**. Questa coda è una linked-list, l'intestazione contiene il puntatore al primo PCB e ogni altro PCB contiene il puntatore al prossimo task della coda.
Esistono anceh altre code, nello specifico, se per esempio un processo deve fare delle lunghe operazioni di I/O, allora il processo che dovrà aspettare verrà messo nella **Wait Queue**.
Un ottimo modo per rappresentare lo scheudling di un processo è tramite un diagramma.

Per prima cosa, un processo viene inserito nella coda ready dove aspettare di essere eseguito, successivamente,in base alla circostanze, può essere messo nel coda di wait o di nuovo nella coda di ready.



(a) Code Ready e Wait.



(b) Diagramma dello Scheudling di un Processo.

Questo ciclo viene eseguito finchè il processo non termina. In tal caso, se il processo ha finito la sua esecuzione, verrà rimosso da ogni coda e le risorse del PCB verranno deallocate.

3.3.2 Cambio di Contesto

In sistemi moderni general-purpose, la frequenza di interrupt che richiedono una routine del kernel per essere gestiti, può essere elevata. Ogni volta che accade, deve essere salvato il **contesto** del processo in esecuzione sul core, per poterlo ripristinare successivamente. Tutti questi dati sono salvati all'interno del PCB, permettendo quindi due azioni: **save state** e **restore state**.

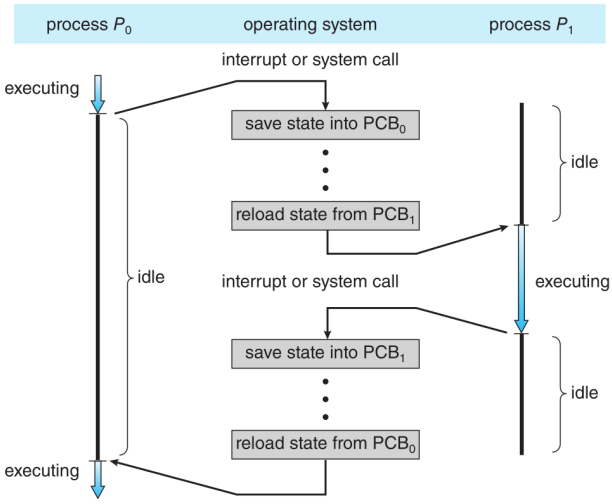


Figura 3.5: Context Switch.

Il tempo per il cambio di contesto è puro overhead, dato che il sistema non fa nulla di utile in questi millisecondi. Più è alta la complessità del sistema e del PCB più tempo sarà necessario. La velocità delle memorie può aiutare a ridurre il tempo di cambio di contesto.

3.4 Creazioni di Processi

I processi possono creare a loro volta altri processi figli, che a loro volta ne possono creare altri. Questo porta alla creazione di un labero dei processi. Ogni processo è indificato dal **Process Identifier** (**PID**), che di solito è un integer. Alla creazione, ci possono essere diverse opzioni:

- **Condivisione delle Risorse:**
  - Genitore e figlio condividono le stesse risorse.
  - Il figlio condivide un sottoinsieme delle risorse del genitore.

  - Genitore e figlio non condividono nessuna risorsa.

- **Esecuzione:**
  - Il processo figlio e quello genitore eseguono contemporaneamene.
  - Il genitore aspetta che il figlio termini.
- **Spazio di Indirizzamento:**
  - Il figlio è una copia duplicata del padre.
  - Viene caricato un altro programma all'interno del figlio.

3.4.1 System Calls `fork()` e `exec()`

Nei sistemi Linux, per creare un nuovo processo può essere usata la funzione `fork()`. Con questa istruzione, viene creato un nuovo processo figlio, con lo stesso spazio di indirizzamento del padre. Al suo interno può essere eseguito anche un altro programma, tramite la funzione `exec()`.
I due processi inizialmente eseguono contemporaneamene, nello specifico, il processo figlio avrà un valore di ritorno di `fork()` con valore uguale a zero, invece, il genitore avrà un valore diverso da zero.

C Esempio di utilizzo di fork() e exec()

```c
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>

int main()
{
  pid_t pid;

  /* fork a child process */
  pid = fork();
  if (pid < 0) { /* error occurred */
    fprintf(stderr, "Fork Failed");
  return 1;
  }
  else if (pid == 0) { /* child process */
    execlp("/bin/ls","ls",NULL);
  }
  else { /* parent process */
    /* parent will wait for the child to complete */
    wait(NULL);
    printf("Child Complete");
  }
}

return 0;
```

3.4.2 Terminazione dei Processi

Normalmente, un processo esegue tutte le istruzioni e poi chiede al sistema operativo di interromperlo con la system call `exit()`. In questo caso, le risorse sono deallocate e viene ritornato un valore al genitore che sta aspettando con la system call `wait()`.
In alcuni casi, è possibile terminare anche con la system call `abort()`.

Se viene terminato un processo genitore, allora anche tutti i figli verranno terminati, altrimenti si possono verficare situazioni problematiche:

- Se il genitore non usa l'istruzione `wait()`, allora i processi figli saranno chiamati **zombie**.
- Se il genitore si interrompe prima di chiamare l'istruzione `wait()`, allora i processi figli saranno **orfani**.

3.5 Comunicazione fra Processi

All'interno di un sistema, i processi possono essere indipendenti oppure possono cooperare. In quest'ultimo caso, necessitano di un modo per comunicare, ossia una **Interprocess Comunication** (**IPC**).
Le modalità possono essere due: **shared memory** o **message passing**.

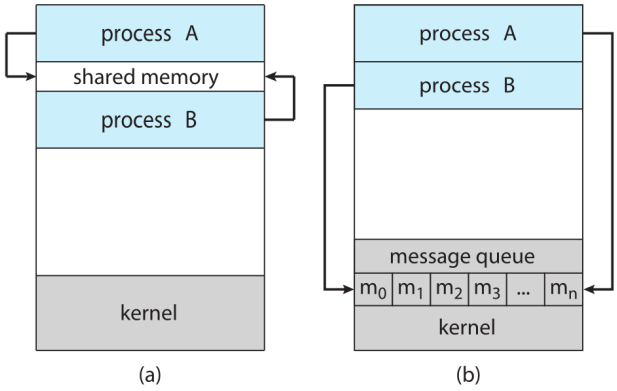


Figura 3.6: Diverse Modalità di Comunicazione.

INTUITION Problema Produttore-Consumatore

Processo **produttore** produce informazione e il processo **consumatore** la consuma.
Può avere due variazioni:

- **Unbounded-Buffer:** non c'è alcun limite pratico alla dimensione del buffer. Il produttore non attende mai. Il consumatore attende se non c'è alcun buffer da consumare.
- **Bounded-Buffer:** presuppone che vi sia una dimensione fissa del buffer. Il produttore deve attendere se tutti i buffer sono pieni. Il consumatore attende se non vi è alcun buffer da consumare.

3.5.1 Shared Memory

Un modo per comunicare fra processi è creare un'area condivisa in cui vengono scritte e lette informazioni. Il controllo è lasciato ai processi utenti e necessita di un meccanismo di sincronizzazione per condinare l'accesso alla memoria.

Per esempio, si potrebbero inserire le seguenti variabili in un'area di memoria condivisa:

C Variabili nella Shared Memory

```c
#define BUFFER_SIZE 10

typedef struct {
  . . .
} item;

item buffer[BUFFER_SIZE];
int in = 0;
int out = 0;
```

Questo metodo è limitato a `BUFFER_SIZE` - 1 elementi.
Si può creare un produttore che scriva dati all'interno di una lista buffer:

C Producer

```c
item next_produced;
```

```c
while (true) {
  /* produce an item in next produced */
  while (((in + 1) % BUFFER_SIZE) == out)
    ; /* do nothing */

  buffer[in] = next_produced;
  in = (in + 1) % BUFFER_SIZE;
}
```

Infine, si può usare un consumatore per leggere i dati da una lista di buffer:

C Consumer

```c
item next_consumed;

while (true) {
  while (in == out)
    ; /* do nothing */

  next_consumed = buffer[out];
  out = (out + 1) % BUFFER_SIZE;
  /* consume the item in next consumed */
}
```

Se si volesse creare una soluzione a questo problema che riesca a riempire tutti i buffer, si può usare un **contatore**. Partendo da 0, quando un buffer viene riempito, il consumatore aumenta di una unità il contatore, invece, quando il consumatore svuota un buffer, il contatore viene diminuito di una unità. Questa soluzione può funzionare, ma se le operazioni non sono atomiche, si può creare una **Race Condition**.

EXAMPLE Race Condition

S0:	[producer] register1 = counter	*register1=5*
S1:	[producer] register1 = register1 + 1	*register1=6*
S2:	[consumer] register2 = counter	*register2=5*
S3:	[consumer] register2 = register2 - 1	*register2=4*
S4:	[producer] counter = register1	*counter=6*
S5:	[consumer] counter = register2	*counter=4*

3.5.2 Messagge Passing

Un altro modo per far comunicare i processi può essere un meccanismo di sincronizzazione per il passaggio di messaggi, senza condividere una stessa area di memoria. In casi in cui i processi siano in computer diversi o reti diverse, questa modalità può risultare molto utile.
Per fare questo servono due operazioni principali: `send(message)` e `receive(messagge)`. Inoltre, serve anche un **canale di comunicazione**, che può avere diverse applicazioni:

- Comunicazioni dirette e indirette.
- Comunicazioni sincrone e asincrone.
- Buffering automatico o esplicito.

Comunicazione Diretta

In questo caso, il link di comunicazione viene stabilito automaticamente fra una coppia di processi e può essere usato solo da essa. In questo caso si possono avere due modalità:

1. **Simmetrica:** entrambi i processi devono nominare l'altro processo per comunicare.
2. **Asimmetrica:** solo chi invia deve nominare il destinatario per comunicare, ma non viceversa.

Comunicazione Indiretta

In questo caso i messaggi vengono inviati a una **mailbox** o **porta**. A livello più astratto può essere visto come un oggetto in cui i messaggi possono essere aggiunti o rimossi. Ogni mailbox dovrà avere il suo numero identificativo.
In questo caso il link di comunicazione si può stabilire solo se entrambi i processi hanno la stessa mailbox, un link può essere associato a più processi.

3.5.3 Sincronizzazione

La comunicazione avviene tramite le primitive `send()` e `receive()` e ci sono diversi design per implementarle:

- **Blocking:** è considerata sincrona. Quindi:
  - **Blocking send:** il processo mittente viene bloccato finché il messaggio non viene ricevuto dal processo destinatario o dalla casella di posta.
  - **Blocking receive:** il ricevente rimane in attesa finché non è disponibile un messaggio.
- **Non Blocking:** è considerata asincrona. Quindi:
  - **Non blocking send:** il processo di invio trasmette il messaggio e riprende il funzionamento.
  - **Non blocking receive:** il ricevente recupera un messaggio valido oppure un valore nullo.

Ci possono essere diverse combinazioni di `send()` e `receive()`, se entrambi sono bloccanti si chiama **rendevouz**.

3.5.4 Buffering

Se la comunicazione è diretta o indiretta, i messaggi scambiati tra processi comunicanti risiedono in una coda temporanea. Queste code possono essere implementate in tre modi:

1. **Capacità zero:** lunghezza pari a zero. Il mandante deve aspettare che il ricevente abbia ricevuto il messaggio (rendevouz).
2. **Capacità limitata:** la lunghezza della coda può contenere n messaggi. Se la coda ha spazio, i messaggi vengono mandati e inseriti, altrimenti, se piena, il mandante deve fermarsi.
3. **Capicità non limitata:** lunghezza della coda praticaticamente illimitata, il mandante può continuare a mandare messaggi senza aspettare.

3.5.5 Esempi POSIX

C Esempio di Producer POSIX

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <sys/shm.h>
#include <sys/stat.h>
#include <sys/mman.h>

int main(){
  /* the size (in bytes) of shm obj */
  const int SIZE = 4096;

  /* name of the shared memory object */
  const char *name = "OS";

  /* strings written to shared memory */
  const char *message_0 = "Hello";
  const char *message_1 = "World!";
```

```c
  /* shared memory file descriptor */
  int fd;

  /* pointer to shared memory obect */
  char *ptr;

  /* create the shared memory object */
  fd = shm_open(name, O_CREAT | O_RDWR, 0666);

  /* configure the size of the shared memory object */
  ftruncate(fd, SIZE);

  /* memory map the shared memory object */
  ptr = (char *)
  mmap(0, SIZE, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);

  /* write to the shared memory object */
  sprintf(ptr,"%s", message_0);
  ptr += strlen(message_0);
  sprintf(ptr,"%s", message_1);
  ptr += strlen(message_1);

  return 0;
}
```

C Esempio POSIX Consumer

```c
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <sys/shm.h>
#include <sys/stat.h>
#include <sys/mman.h>

int main(){
  /* the size (in bytes) of shared memory object */
  const int SIZE = 4096;

  /* name of the shared memory object */
  const char *name = "OS";

  /* shared memory file descriptor */
  int fd;

  /* pointer to shared memory obect */
  char *ptr;

  /* open the shared memory object */
  fd = shm_open(name, O_RDONLY, 0666);

  /* memory map the shared memory object */
  ptr = (char *)
  mmap(0, SIZE, PROT_READ, MAP_SHARED, fd, 0);

  /* read from the shared memory object */
  printf("%s",(char *)ptr);

  /* remove the shared memory object */
  shm_unlink(name);

  return 0;
}
```